

Threat Model — public-safety-alerts-aggregator

Summary and Remediation Roadmap

Author: Pradip | Date: 23 August 2026

Summary

- What this is: A Node.js + Next.js application that collects public safety alerts from external sources (NSWRFS, SES, Bureau of Meteorology, Transport NSW, etc.), stores them in a local database, and exposes a web dashboard and APIs for searching and visualizing alerts.
- Key runtime pieces (evidence): backend/server.js (Express server, background schedulers & notificationprocessor), backend/services/* (ingest, maintenance, scheduler), backend/data-collection/api_collectors/* (external API collectors such as tfnswCollector.js), backend/database/* (DAOs), client/src/app/page.tsx (Next.js App Router page that fetches /alerts and passes a Google Maps API key to a map component).

Assets and Sensitive Items

Secrets / credentials (present in code as environment variables):

- TFNSW_API_KEY (used by backend collectors / TFNSW requests) —backend/data-collection/api_collectors/tfnswCollector.js, backend/controllers/alertsController.js
- GOOGLE_MAPS_API_KEY (used by client page: client/src/app/page.tsx)
- process.env.* values used for scheduler/maintenance (MAINTENANCE_CRON, ALERT_RETENTION_DAYS, NOTIFICATION_RETENTION_DAYS)
- Data store: local database (backend/database/db + DAO files). Likely SQLite / file-backed DB from projectstructure.
- Endpoints/APIs: /alerts, /sources, /geo (backend/routes), plus any admin/maintenance endpoints not yet visible.
- Background processes: Ingest orchestrator and maintenance scheduler (backend/services/SchedulerService.js, NotificationService.startProcessing in server.js).
- External dependencies: multiple external APIs (RFS, SES, TfNSW, BOM, Google Maps) and npm packages (Express, cors, node-cron, dotenv, fetch usage).

Architecture & Data Flows

- Ingest flow: SchedulerService schedules IngestOrchestratorService.autodiscoverAndRun -> data collectors (backend/data-collection/api_collectors/*.js) call external APIs using api keys -> collectors transform features -> DAOs persist alerts to local DB.
- API surface: Express server listens (backend/server.js) and exposes /alerts, /sources, /geo routes; controllers fetch from DB and return JSON to client.
- Client: Next.js page (client/src/app/page.tsx) fetches backend endpoints (http://localhost:3001/alerts) and renders maps/components using a Google Maps API key passed into AlertMap.

Primary Attack Surface

- Public HTTP endpoints (/alerts, /sources, /geo) — injection, DoS, excessive scraping, unauthenticated access to data.
- Background collectors that fetch external URLs and parse responses — SSRF-like risks, dependency on upstream data integrity, injection of malicious content into stored alerts.

- Database (local file) — confidentiality/integrity if host filesystem accessible.
- Secrets in environment variables — risk of exposure in client bundle or logs.
- Client-side rendering of external content (titles, descriptions, links) — stored or reflected XSS if output is not encoded/escaped.
- Scheduler/maintenance features that execute periodic tasks with cron expressions from environment — risk of misconfiguration and privilege escalation of background tasks.

Threats, Likelihood & Impact, and Evidence

Each threat includes references to repository files where it appears.

1) Secret exposure in client bundle — Likelihood: Medium. Impact: High.

Evidence: `client/src/app/page.tsx` reads `process.env.GOOGLE_MAPS_API_KEY` and passes it into `AlertMap` component. In `Next.js`, environment variables can end up in the client bundle if not using the proper `NEXT_PUBLIC_*` rules or if server-side code is mixed into client components.

Risk: leaking Google Maps API key or other keys to public, enabling abuse (quota theft, billing).

Mitigation (priority): Ensure only public-safe keys use `NEXT_PUBLIC_` prefix and that sensitive keys remain server-only; confirm build config and move usages to server endpoints that proxy map metadata when possible. Restrict API key to allowed referrers (domain restrictions) in Google Cloud.

2) Insecure or overly permissive CORS / open endpoints — Likelihood: Medium. Impact:

Medium.

Evidence: `backend/server.js` uses `cors(corsOptions)` with config in `backend/config/corsOptions` (not shown) — needs review. Client fetch uses plain `http://localhost:3001` in code which indicates no enforced TLS in dev.

Risk: if `corsOptions` allows `*`, any origin can read responses; combined with sensitive data it can enable exfiltration from other origins.

Mitigation: Restrict CORS to known front-end origins; require authentication for any admin or sensitive endpoints; use HTTPS in deployment.

3) Stored XSS from external feed fields — Likelihood: Medium. Impact: High (on UI consumers).

Evidence: Collectors and controllers accept fields such as title, description, headline, advice from external feeds (`tfnswCollector.js` and `alertsController.js`) and store them. Client renders alerts on maps/UI.

Risk: Upstream feeds may contain malicious HTML/JS which, if stored and sent to browsers and rendered without safe encoding, leads to stored XSS.

Mitigation: Sanitize or strip HTML before storing, or encode/escape on render. Use a vetted sanitizer library on server-side ingestion and also validate on output.

4) SQL injection or unsafe dynamic SQL — Likelihood: Low–Medium. Impact: High.

Evidence: `MaintenanceService._buildThreshold` returns a datetime SQL fragment that is interpolated into queries in `getExpiredAlertIds`; some DAOs appear to be called with programmatic table names or condition strings. If any parts accept untrusted input and concatenate into SQL, injection is possible.

Risk: Manipulated env vars or inputs could alter SQL semantics, remove/modify data.

Mitigation: Use parameterized queries everywhere. Avoid concatenating user/externally-influenced values into SQL strings. Validate/parse numeric env vars before use.

5) API key misuse & supply-chain abuse — Likelihood: Medium. Impact: Medium.

Evidence: Collectors fetch external APIs using `process.env.TFNSW_API_KEY` and others. The system persists external data into DB and exposes it via API.

Risk: If external APIs return malicious, malformed, or spoofed content, app could store or display misleading information or be used as an attack vector.

Mitigation: Validate fetched data schema, strict JSON parsing, limit and sanitize fields persisted; monitor external responses for anomalies; separate raw feed storage from canonical sanitized records.

6) Denial-of-Service (ingest & scheduler) — Likelihood: Medium. Impact: Medium–High.

Evidence: SchedulerService schedules periodic jobs (ingest, maintenance) that may perform heavy work (autodiscoverAndRun). There is no visible rate limiting or backoff.

Risk: High-frequency failures or runaway collectors could consume CPU, memory, DB locks, or overflow storage.

Mitigation: Add concurrency limits, timeouts, exponential backoff for failed upstream calls, circuit breakers, and resource monitoring; ensure maintenance tasks use transactions with limits.

7) Local DB file exposure & insecure file permissions — Likelihood: Low–Medium. Impact: High.

Evidence: backend/database is local; likely a file-backed DB (SQLite). Logs/DB directories exist under repo (backend/logs, backend/database).

Risk: If host filesystem is not protected, DB can be copied or modified.

Mitigation: Run DB under a dedicated service with least-privilege, secure file permissions, and backups encrypted at rest.

8) Unauthenticated API -> data tampering & scraping — Likelihood: High. Impact: Medium.

Evidence: /alerts endpoint is publicly available and used by client without auth. There is no visible auth or rate limiting for API consumers.

Risk: Attackers can scrape data, or if there are write endpoints (collectors or webhooks) they could try injection.

Mitigation: Add rate limiting, API keys, or at least usage quotas; protect write endpoints; consider read auth for sensitive data.

9) Dependency & supply-chain vulnerabilities — Likelihood: Medium. Impact: Medium–High.

Evidence: Project uses many npm modules (Express, node-cron, fetch, better-sqlite3, etc.). package-lock files are present.

Risk: Known npm vulnerabilities in outdated packages can be exploited.

Mitigation: Pin and regularly audit dependencies (npm audit, Snyk), run automated dependency scanning in CI, keep lockfiles up-to-date.

10) Log injection & sensitive info in logs — Likelihood: Medium. Impact: Medium.

Evidence: middleware/logEvents logger used (backend/server.js). External data fields are logged and may include untrusted content.

Risk: Log files can contain injected content or secrets that are later viewed by humans or stored in log management.

Mitigation: Escape/clean logged content; avoid logging secrets; centralize logs securely and enable rotation.

Concrete Prioritized Remediation Actions (Roadmap)

1) Secrets & build config (High priority)

- Ensure no secret API keys are exposed to client bundles. Move `GOOGLE_MAPS_API_KEY` usage to server-side or use properly-scoped public key prefixes (`NEXT_PUBLIC_`) only for intentionally public keys. Enforce key restrictions (HTTP referrers, IPs).
- Add a secrets-management policy (do not commit `.env`). Add CI pre-commit checks to block secrets.

2) Input/output hardening (High priority)

- Sanitize and validate all external feed fields at ingestion (server-side). Use a well-maintained sanitizer when storing/returning descriptions/headlines.
- Escape/encode when rendering in the browser.
- Enforce JSON schema validation for collectors' parsed objects (ajv or equivalent).

3) Database & SQL safety (High priority)

- Audit all DAO code to ensure parameterized queries; remove any place that interpolates untrusted data into SQL strings. Validate numeric env vars (`ALERT_RETENTION_DAYS`, etc.) before use.
- Restrict DB file permissions and configure backups/encryption as appropriate.

4) Network & API security (High priority)

- Enforce HTTPS everywhere (client-app <-> backend). Avoid using `http://localhost:3001` in production code. Configure TLS and reverse proxy.
- Harden CORS: restrict allowed origins in backend/config/corsOptions.

5) Rate limiting & auth (Medium–High priority)

- Add rate limiting (express-rate-limit, or gateway/WAF) to public endpoints.
- Consider API keys or token-based access for heavy endpoints and admin operations.

6) Scheduler and ingestion hardening (Medium priority)

- Add timeouts and concurrency limits for collectors (abortable fetches).
- Add monitoring & alerts for scheduler failures, long-running tasks, and out-of-bounds DB growth.

7) Dependency & supply-chain (Medium priority)

- Add automatic dependency scanning and a scheduled update process. Pin versions where appropriate; update common vulnerable packages.

8) Observability & incident response (Medium priority)

- Add structured logging, error metrics, monitoring (Prometheus/Cloud provider metrics), and alerting for unusual ingestion or scheduler behavior.
- Maintain an incident response checklist: identify affected data, rotate compromised keys, restore from backups, revoke cloud API keys if needed.

Detection & Monitoring Suggestions

- Alert on abnormal increases in ingest volume, unexpected new sources discovered by orchestrator, or JSON schema validation failures.
- Log and monitor 4xx/5xx response spikes and long response times for /alerts and ingestion tasks.
- Periodically scan DB for suspicious payloads (script tags in title/description).

Privacy & Compliance Notes

- Alerts include location and regional metadata. Evaluate whether any stored payload contains PII and apply appropriate retention and minimization (MaintenanceService exists — ensure retention policy enforced).
- Ensure secure data deletion semantics when maintenance cleanup runs.

Risk Matrix (Selected Items)

Item	Likelihood	Impact	Priority
Secret exposure (client or logs)	Medium	High	Immediate
Stored XSS from feeds	Medium	High	Immediate
SQL injection via dynamic SQL	Low–Medium	High	High
DoS via ingestion/scheduler	Medium	Medium–High	High
Dependency vulnerabilities	Medium	Medium	Medium

References to Repository Files (for Reviewers / Devs)

- Server & startup: backend/server.js
- Schedulers: backend/services/SchedulerService.js
- Maintenance/cleanup: backend/services/MaintenanceService.js
- Collectors example: backend/data-collection/api_collectors/tfnswCollector.js
- API controllers: backend/controllers/alertsController.js
- Client entry: client/src/app/page.tsx
- Config: backend/config/corsOptions (check actual cors rules)
- DB & DAOs: backend/database/* (audit all DAO implementations)